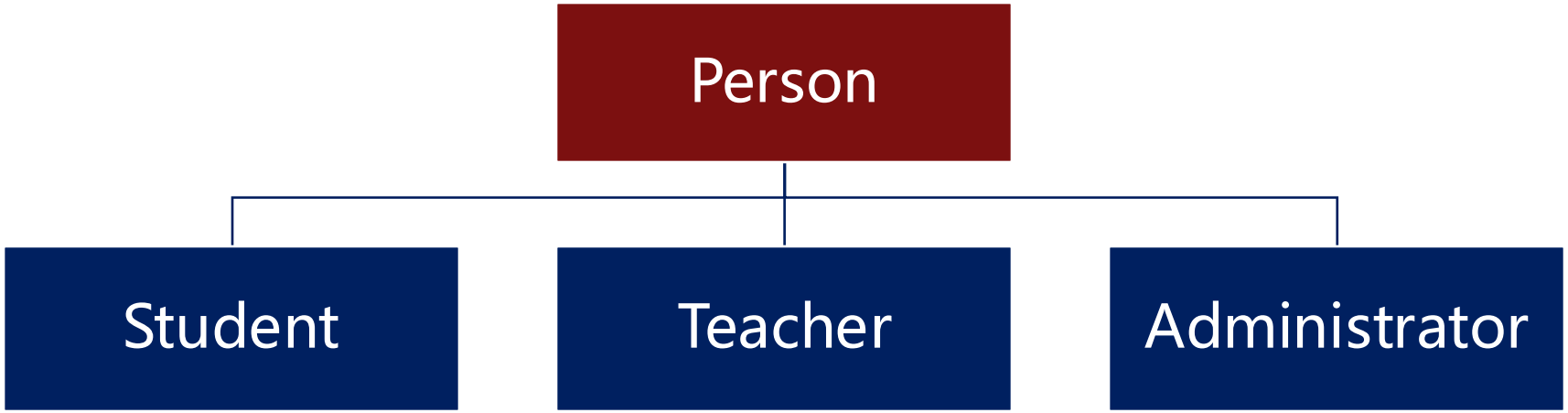
Chapter 4 Interfaces and Inheritance

Outline

- **Inheritance**
- **Interfaces and abstract classes**
- **Polymorphism and rewriting**
- **Java multi-inheritance issues**
- **Superclass vs. super keyword**
- **Inheritance framework for exceptions**

Inheritance - Superclass and subclasses



- The Person class is related to other classes, such as the Student class
- There is an obvious is-a relationship between Student and Person, which is a clear feature of inheritance

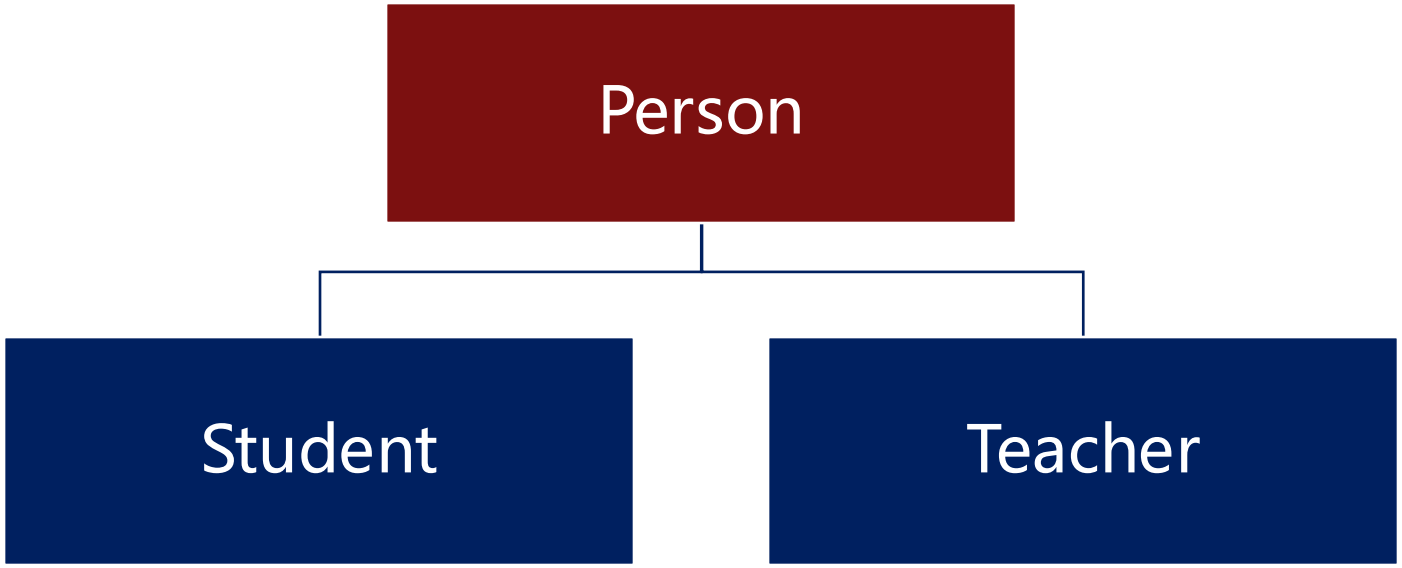
Inheritance - classes, superclasses, and subclasses

- The keyword “extends” indicates that a new class being constructed derives from an existing class.
  - Existing classes: superclass, base class, parent class
  - New classes: subclasses, derivatives, children
- Inheritance reuses the methods of existing classes and adds new methods and fields to adapt the new classes to new situations.
- Parent/superclasses describe the properties and methods that are shared by the subclasses.
- Different subclasses have their own different properties and methods

Inheritance

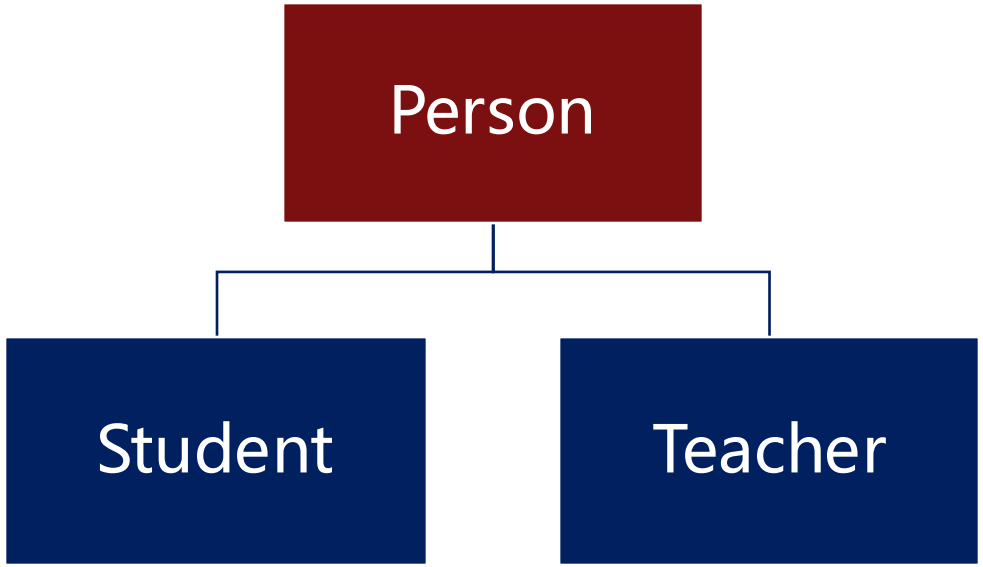
- Inheritance is one of the three important features of object-oriented programming
- Advantages
  - Improve code reusability
  - Improve the scalability of the program
  - This creates a relationship between classes and forms the basis of polymorphism
- Disadvantages
  - Increase class coupling (changes in one class affect other related classes)

Abstract Class



- The parent class extracts the properties and methods that are common to the child classes.
- If any of these properties and methods are undecided, then we can define them as abstract methods and implement them in the later subclasses.
- To put it simply, a class with an abstract method is an abstract class.

Abstract Class - Define an abstract class




```
1. //Define an abstract class
2. public abstract class Person{
3.     //Ordinary method
4.     public String getName()
5.     {
6.         return name;
7.     }
8.     //Abstract approach
9.     //There is no method body, and it
   is decorated with abstract
10.     public abstract void getMission();
11.}
12.
13.Person P = new Person();
14.// Compilation error, abstract class
   cannot be instantiated
```

Abstract Classes - The role of abstract classes

- Abstract classes cannot be instantiated
- function：
  - An abstract approach is actually equivalent to defining a "specification"
  - It can only be inherited, and the subclass implements its defined abstract methods
  - Can be used to implement polymorphism

Interface

- In abstract classes, the abstract method essentially defines the interface specification to ensure the subclasses all have the same interface implementation
- Interface:
  - More abstract than an abstract class: without attributes, all methods are abstract

```
1. //Define an abstract class
2. public abstract class Person{
3.     //Abstract method
4.     public abstract void getDuty();
5.     public abstract void getMission();
6. }
```

```
1. //Define an interface
2. interface Person{
3.     //Abstract method
4.     void getDuty();
5.     void getMission();
6. }
```

Interface

- When a specific class implements an interface, the implements keyword is used

```
public class Student implements Person{
    private String name;
    private int age;
    @Override
    public void getDuty(){
        System.out.println("Study");
    }
    @Override
    public void getMission(){
        System.out.println("Study hard");
    }
}
```

Interface VS. Abstract classes

- Interface vs. abstract class

	Abstract class	Interface
Abstract method	Abstract methods can define	Abstract methods can define
Field	Abstract methods can define	No fields
Inheritance	Can only extends one class	Can implements many interfaces

- Why do you need interfaces when you have abstract classes?
  - Abstract classes don't solve the problem of multiple inheritance

Polymorphism

- Polymorphism is one of the three important features of object-oriented programming
- Polymorphism is the ability of the same behavior to have multiple manifestations
  - Different ways to deal with it in different situations
  - The variables and methods defined in a program are not determined at the time of programming, but only during the program's run.

Polymorphism

- **Benefits**
  - Reduce coupling
  - Enhance replaceability
  - Enhance scalability
  - Increase flexibility
- **Three necessary conditions：**
  - inherit
  - override
  - The parent class reference points to the child class
- **Three ways to implement polymorphism：**
  - **Override**
  - **Abstract Classes and Abstract Methods**
  - **Interface**

Implementation 1: Override

- Some methods in the parent class don't necessarily work for the child class. This method needs to be overridden in the parent class.
- Override: If you define a method in a child class with the same name, parameters, and return type as a method in the parent class, then you can say that the method of the child class overrides the method of the parent class.

Implementation 1: Override

```
1. public class Person{
2.     private String name;
3.     private int age;

4.     public void getTarget(){
5.         System.out.println( “The good
   life” );
6.     }
7. }
```

```
1. public class Student extends Person{
2.     @Override
       public void getTarget(){
3.         System.out.println(“Learn");
4.     }
5. }
```

```
1. public class Teacher extends Person{
2.     @Override
       public void getTarget(){
3.         System.out.println(“Teach");
4.     }
5. }
```

- In the Student and Teacher classes, we've overridden the getTarget method to override the previous getTarget.
- These methods have the same name, return type, and parameters.
- Programming skills: add @Override allows the compiler to help check if the correct override has been made.

Implementation 1: Override

```
public static void show_target(Person P_x) {
     System.out.print(" target: ");
     P_x.getTarget();
 }
Person P1= new Student();
Person P2= new Teacher();
show_target(P1);
show_target(P2);
```

- The method of the child class override is invoked by reference to the parent class
- In the show_target() function, we only need to pass in a reference to the parent class
  - **Super_class_name reference_name = new subclass_name();**

Implementation 2: abstract classes

```
1. //Define an abstract class
2. public abstract class Person{
3.     ……
4.     //Abstract approach
5.     //There is no method body, and it
   is decorated with abstract
6.     public abstract void getTarget();
7.     ……
8. }
```

Implementation 2: abstract classes

```
public class Engineer extends Person{
    @Override
    public void getTarget(){
        System.out.println(“Technology
changes the world");
    }
}
```

```
public class Scientist extends Person{
    @Override
    public void getTarget(){
        System.out.println(“Climb the
heights of science");
    }
}
```

```
public static void main(String[] args) {
    Person Wang = new Engineer();
    Wang.getTarget();
    Person Li = new Scientist();
    Li.getTarget();
}
```

- Definition format for polymorphism:
- Superclass_name reference_name = new subclass_name ();

```
Person Wang = new Engineer();
```

Implementation 3: interfaces

```
//Interface: Target
public interface GetTarget(){
    void getTarget();
}
```

An interface is a pure abstract class that is more abstract than a normal abstract class, with no attributes, only methods.

Implementation 3: interfaces

```
//Talent class implements the target interface
public class Talent implements GetTarget(){
    @Override
    public void getTarget(){
        System.out.println(“Build the most important facilities for
the country");
    }
}
```

```
// Entrepreneur class implements the target interface
public class  Entrepreneur implements GetTarget(){
    @Override
    public void getTarget(){
        System.out.println(“Promote common prosperity");
    }
}
```

Implementation 3: interfaces

```
public static void main(String[] args) {
    GetTarget  Liu = new Talent();
    Liu.getTarget();
    GetTarget  Ma = new Entrepreneur();
    Ma.getTarget();
}
```

The objects Liu and Ma here can be considered as subclasses of the interface GetTarget

Does Java allow multiple inheritances

- Multiple inheritance means that a class can inherit behaviors and characteristics from multiple parent classes at the same time, whereas Java only allows single inheritance to ensure data security.
- Problems with multiple inheritance:
  - If a child class inherits from a parent class that has the same member variable, the child class will not be able to tell which parent class's member variable to use when referencing the variable.
  - If a child class inherits from multiple parent classes that have the same method, and the child class does not override the method (if it does, it uses the method in the child class), then it will not be possible to determine which parent class's method is called when the method is called.

How Java implements multi-inheritance effects

- Sometimes developers do need to implement multiple inheritance, and this is the case in real life. Heredity, for example, we can inherit both the father's and the mother's behaviors and traits.
- Java provides two ways to implement multiple inheritance：
  - Inner classes
  - Interface

Multi-inheritance 1: Inner class

- Inner Classes: Inside a class, you can define not only member variables and methods, but also another class. If you define another class B inside class A, class B is called an inner class, and class A is called an outer class.
  - The inner class can be well hidden, and you have access to all elements of the outer class.

Multi-inheritance 1: Inner class implementation

- We will introduce how children can use internal classes to inherit the good genes of both their father and mother using common examples of heredity in their lives.
- We create the Father class, add the strong() method to the class; Again, create the Mother class and add the smart() method to the class, as follows:

```
public class Father{
    public int strong(){
        // Strength Index
        return 9;
    }
}
```

```
public class Mother{
    public int smart(){
        // Intelligence Index
        return 8;
    }
}
```

Multi-inheritance 1: Inner class implementation

- Then, create a Son class in which you can implement multiple inheritance effects with an inner class. The code looks like this on the right:
  - The Son class defines two inner classes that inherit from the Father class and the Mother class, respectively, and both get the behavior of their respective parent classes.

```
1. public class Son {
2.     // Inner class inherits from the Father class
3.     class Father_Inner extends Father {
4.         public int strong() {
5.             return super.strong() + 1;
6.         }
7.     }
8.     // Inner class inherits the Mother class
9.     class Mother_Inner extends Mother {
10.        public int smart() {
11.            return super.smart() + 2;
12.        }
13.    }
14.     public int getStrong() {
15.         return new Father_Inner().strong();
16.     }
17.     public int getSmart() {
18.         return new Mother_Inner().smart();
19.     }
20. }
```

Multi-inheritance 2: Implemented with interfaces

- First of all， define interfaces Father and Mother. And then, we can use Daughter to inherit from Father and Mother。
  - In Java, a class can inherit only one class, but an interface can inherit multiple interfaces.

```
1. public interface Father{
2.     public void strong();
3. }
4.
5. public interface Mother{
6.     public void smart();
7. }
8.
9. public interface Daughter extends Father, Mother{
10.    public void kind();
11.}
```

Multi-inheritance 2: Implemented with interfaces

Then, the Girl class implements the interface Daughter, and rewrites the strong method of Father, the smart method of Mother, and the kind method of Daughter in the class.

```
1. public class Girl implements Daughter{
2.     public static void main(String[] args){
3.     }
4.     @Override
5.     public void strong(){
6.         System.out.println(“She’s not strong.");
7.     }
8.     @Override
9.     public void smart(){
10.         System.out.println(“She’s very smart.");
11.     }
12.     @Override
13.     public void kind(){
14.         System.out.println(“She’s very kind.")
15.     }
16.}
```

equals method

- The equals() method in the Object class is used to compare whether two objects are equal.
- The equals() method compares two objects to determine whether the two object references point to the same object.

```
1.  class IfStudentEqual {
2.      public static void main(String[] args) {
3.          // Create two objects
4.          Object Student_1 = new Student("Zhangsan",19);
5.          Object Student_2 = new Student("Lisi",18);
6.
            // Different objects, with different memory addresses, are not equal
7.          System.out.println(Student_1.equals(Student_2));
8.
            // Object references, with the same memory addresses, are equal
9.          Object Student_3 = Student_1;
10.         System.out.println(Student_1.equals(Student_3));
11.     }
12. }
```

super keyword

- The functionality of the super keyword：
  - Explicitly call the parent class constructor in the constructor of the child class.
  - Access member methods and variables of the parent class.

Super calls the parent class constructor

- The super keyword can explicitly call the constructor of the parent class in the constructor of the subclass, in a basic format such as: `super(parameters);`
  - where parameter-list specifies all the parameters in the parent class constructor.

```
1. public class Person {
2.     public Person(String name, int age) {
3.     }
4. }
5. public class Student extends Person {
6.     public Student(String name, int age, String school) {
7.         super(name, age); // Call a constructor with 2
   parameters in the parent class
8.     }
9. }
```

Super access to the parent class member

- When a member variable or method of a child class has the same name as the parent class, for example, the child class overrides a method of the parent class, we can use the super keyword to call the method in the parent class.
- Accessing members in a parent class using super is similar to the use of the this keyword, except that it references the parent class of the child class, syntactically like : `super.X`
  - where X is a property or method in the parent class.

Super access to parent class member variables

- In the example on the right, both the parent and child classes have a member variable age. We can use the super keyword to access the age variable in the Person class.

```
1.  class Person {
2.      int age = 39;
3.  }
4.  class Student extends Person {
5.      int age = 18;
6.      void display() {
7.          System.out.println(”Student Age: " + super.age);
8.      }
9.  }
10. class Test {
11.     public static void main(String[] args) {
12.         Student stu = new Student();
13.         stu.display();
14.     }
15. }
```

Is the output? ⇨ Student Age：39

super calls the parent class member method

- When both the parent and child classes have the same method name, you can use the super keyword to access the parent class's methods.

```
1.  class Person {
2.      void getTarget() {
3.          System.out.println("A good life");
4.      }
5.  }
6.  class Student extends Person {
7.      void getTarget() {
8.          System.out.println("High scores");
9.      }
10.     void display() {
11.         getTarget()
12.         super.getTarget();
13.     }
14. }
```

Call getTarget() from Student Class

Call getTarget() from Person Class